

AWS Developer Exam: Domain 3 – Test Applications in Development Environments

Overview

Testing deployed code is a critical part of building a well-architected application on AWS. This task statement covers:

- Deployment strategies and when to use each

- AWS services that perform application deployment

- Testing with API Gateway mock integrations and development stages

- Step Functions Local for state machine testing

- Lambda versioning and aliases for managing deployments across environments

Designing a Deployment Solution

When designing a deployment solution, consider each step of the application lifecycle:

- Provision** — How infrastructure is created

- Configure** — How resources are set up

- Deploy** — How application code is delivered

- Scale** — How the system handles load changes

- Monitor** — How you observe and respond to system behavior

The deployment process you choose depends on your desired balance of:

- Control

- Speed

- Cost

- Risk tolerance

AWS Deployment Services

Multiple AWS services perform application deployment, each with different capabilities:

Service	Best For
AWS CodeDeploy	Automated deployments to EC2, on-premises, Lambda, and ECS
AWS CloudFormation	Infrastructure as code; managing the general infrastructure layer
AWS Elastic Beanstalk	Simplified application deployment with managed infrastructure
Amazon ECS	Containerized application deployments
Amazon EKS	Kubernetes-based container deployments
AWS OpsWorks	Chef/Puppet-based configuration management and deployments
Amazon S3 + CloudFront	Static website and content deployments
Best Practice: Use CloudFormation to manage general infrastructure and a more specialized deployment solution (like CodeDeploy) for managing application updates. These can be combined for complete deployment functionality.	

Deployment Strategies

In-Place Deployment

The application is **stopped**, the latest version is **installed**, and then the new version is **started and validated**. The existing instances are updated directly.
Use when updating within the same runtime and platform version.

Blue/Green Deployment

Two environments are maintained: **blue** (current production) and **green** (new version). Traffic is switched from blue to green once the new version is validated.
Best choice when updating with a different runtime, server version, or major platform version.
Reduces risk — you can roll back instantly by switching traffic back to blue.

Rolling Deployment

Updates are deployed to a **subset of instances** at a time.
Reduces downtime but takes longer than an all-at-once deployment.
Good balance between speed and risk.

All-at-Once Deployment

Updates are deployed to **all instances simultaneously**.
Fastest deployment method but carries the highest risk — all instances are affected at the same time.

Combining Deployments

Use CloudFormation for infrastructure management alongside a specialized tool (e.g., CodeDeploy) for application updates. This gives you the flexibility and control of both approaches.

Optimizing EC2 Deployments

For applications that rely on customizing or deploying on EC2 instances, optimize deployments using two strategies:

Bootstrapping — Run scripts at instance launch to install and configure software dynamically.

Prebaking AMIs — Pre-install software and configurations into an AMI so instances launch faster with less bootstrapping needed at runtime.

CloudFormation Helper Scripts

CloudFormation includes helper scripts based on `cloud-init`. Key helper to know:

Helper Script	Purpose
<code>cfn-init</code>	Retrieve metadata, install packages, start services, and create files on EC2 instances

Exam Tip: If asked what you use to retrieve metadata, install packages, start services, or create files from a CloudFormation template — the answer is **cfn-init**.

Deploying SAM Templates to Staging Environments

`sam deploy` deploys an AWS SAM application. By default, the SAM CLI assumes you are working in the **current directory** (your project root).

To override this and deploy a specific template to a different staging environment, use the `--template` option:

```
sam deploy --template <path-to-template>
```

This tells SAM to deploy only that specified template and the local resources it points to, rather than the default project template.

Testing with API Gateway

Mock Integrations

When services behind an API Gateway are not set up the same as the services consuming the APIs, you can test all API responses using **mock integrations** to ensure no issues arise with consumers.

Amazon API Gateway supports:

Simple mock integrations — Return a fixed response for an API method without calling a backend.

Complex mock integrations — Include conditional status codes based on request headers, simulating different response scenarios.

Development Endpoints and Stages

After the initial deployment of an API, you can add additional **stages** in API Gateway for testing. Stage settings allow you to:

Enable **caching**

Customize **request throttling**

Configure **logging**

Define **stage variables** (configuration attributes associated with a deployment stage)

Attach a **canary release** for gradual traffic shifting and testing

Stage variables act as configuration attributes scoped to a specific deployment stage — useful for pointing different stages at different backend resources (e.g., different Lambda function aliases or different DB endpoints).

Testing Step Functions with Step Functions Local

Step Functions Local allows you to test the execution paths of your state machines **without actually calling integrated services**.

How It Works

Create a **mock configuration file** that contains:

The desired output of your service integrations as **mocked responses**

Test case executions that use those mocked responses to simulate execution paths

Provide the mock configuration file to Step Functions Local.

Run state machines using the mocked responses instead of making actual service integration calls.

This allows you to test logic and execution paths in isolation without incurring costs or side effects from real service calls.

Lambda Versioning and Aliases

Managing different versions of Lambda functions across development, beta, and production environments requires a structured approach. Lambda versioning and aliases provide that structure.

Lambda Versions

You can **publish one or more versions** of a Lambda function.

The `$LATEST` version always reflects your most recent unpublished changes.

Published versions are **immutable** — once published, a version's code and configuration cannot be changed.

Typical Workflow Across Environments

Make code changes to `$LATEST`.

Publish a **new version** (e.g., v4).

Deploy v4 to **dev** for initial testing.

Promote to **beta** for further testing.

Promote to **prod** when validated.

The Problem with Versions Alone

Every time you promote a new version to production, any service that invokes your Lambda function (e.g., an S3 event trigger) must be updated with the **new version ARN** — because version ARNs are immutable and version-specific.

Lambda Aliases — The Solution

An **alias** is a pointer to a specific version of a Lambda function. Key characteristics:

Aliases **can be changed** to point to different versions at any time.

Aliases have their **own ARN** — stable and unchanging.

Event sources (e.g., S3, API Gateway, DynamoDB) reference the **alias ARN**, not the version ARN.

How Aliases Work in Practice

Element	Role
S3 Event Source	Configured with the alias ARN (e.g., the <code>prod</code> alias ARN) — never changes
<code>prod</code> Alias	Points to the current production version (e.g., v3)
Promotion	Update the <code>prod</code> alias to point to the new version (e.g., v4) — S3 config is untouched

Result: The S3 configuration never needs to be updated. Changing which version the alias points to is all it takes to promote a new version to production.

Additional Alias Capabilities

Weighted routing — Split traffic between two versions using an alias (e.g., 90% to v3, 10% to v4 for canary testing).

Environment abstraction — Each environment (dev, beta, prod) can have its own alias pointing to the appropriate version.

Lambda Environments and Versioning Summary

Concept	Description
<code>\$LATEST</code>	Always the most recent unpublished code; mutable
Published Version	Immutable snapshot of code + configuration; has a numbered ARN
Alias	Named pointer to a specific version; mutable and stable ARN
Weighted Alias	Splits traffic between two versions for canary/gradual rollouts

Key Exam Tips

All-at-once deployments are the fastest. **Blue/green** is best for major version or runtime changes.

Blue/green allows instant rollback by switching traffic back — lowest risk for major updates.

cfn-init is the CloudFormation helper script for installing packages, starting services, retrieving metadata, and creating files on EC2.

Use `sam deploy --template <path>` to deploy a specific SAM template to a staging environment other than the current directory default.

API Gateway mock integrations allow you to test API responses without a real backend — supports both simple fixed responses and complex conditional responses.

Stage variables in API Gateway act as environment-specific configuration attributes — use them to point stages at different backends without changing code.

Step Functions Local + mock config file = test state machine execution paths without calling real services.

Lambda versions are immutable — once published, they cannot be changed.

Lambda aliases abstract version promotion — event sources reference the alias ARN, so promoting a new version only requires updating the alias, not the event source configuration.

Weighted aliases enable canary deployments for Lambda — gradually shift traffic from one version to another. EOF